



南京大學

本科畢業論文

学 院	智能软件与工程学院
专 业	软件工程（智能化软件）
题 目	实践中的理论最优哈希表
年 级	2022 学 号 221900323
学生姓名	纪泽操
指导教师	刘明谋 职 称 副教授
提交日期	2026 年 5 月 20 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：实践中的理论最优哈希表

学院：智能软件与工程学院

专业：软件工程（智能化软件）

本科生姓名：纪泽操

指导教师（姓名、职称）：刘明谋 副教授

摘要：

哈希表是高性能存储系统中最基础的数据结构之一。为在高负载下同时获得接近常数的查询代价与较高的空间利用率，本文面向论文《On the Optimal Time/Space Tradeoff for Hash Tables》提出的结构，完成其在工程环境中的复现与系统化实现。该结构以全局分桶（bin）与桶内分块（mini-bin）组织槽位，并为每个桶设置常数大小的回退区（fallback）；对每个键仅需检查“偏好 mini-bin 区间”和“fallback 区间”两段连续下标范围即可完成查询，从而在工程意义上将一次查询约束为 1-2 次区间级探测。为处理冲突，本文实现了有界深度的置换插入（k-kick），在深度超过上界时将元素留在回退区以保证插入过程可终止。

系统实现采用 MySQL 作为存储与并发控制载体，将槽位映射为以全局下标为主键的关系表行，并存储 64 位键值与 16 位指纹以加速过滤；后端使用 C++20 构建 HTTP 服务，前端基于 Vue 3 提供交互式实验界面，支持初始化、插入、查询、删除、在线扩容以及探针次数、回退占比与踢出深度分布等统计接口。实验与工程实践表明，该实现能够在较高负载因子下保持稳定的查询探测次数，并通过对在线扩容机制维持长期运行的可用性。本文工作为理论最优时间/空间折中哈希表在数据库与服务化场景中的工程落地提供了可复现的参考实现与实验支撑。

关键词：哈希表；时间空间折中；数据库存储；指纹；动态扩容

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: On the Optimal Time/Space Tradeoff for Hash Tables

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software Engineering (Intelligent Software)

UNDERGRADUATE: Ji Zecao

MENTOR: Liu Mingmou, Associate Professor

ABSTRACT:

Hash tables are a fundamental building block in high-performance storage systems. To achieve near-constant lookup cost under high load while maintaining good space efficiency, this thesis reproduces and engineers the core design proposed in *On the Optimal Time/Space Tradeoff for Hash Tables*. The data layout partitions slots into global bins and intra-bin mini-bins, and augments each bin with a constant-size fallback area. For any key, a lookup only scans two contiguous index ranges—the preferred mini-bin segment and the bin’s fallback segment—so a query is bounded to about one to two range-level probes in practice. To resolve collisions during insertion, we implement bounded-depth displacement (k-kick); when the recursion depth exceeds the preset limit, the key is kept in the fallback path to guarantee termination.

Our system uses MySQL as the persistent storage and concurrency control substrate, representing the logical slot array as table rows keyed by a global index. Each occupied slot stores the full 64-bit key/value and a 16-bit fingerprint for efficient filtering. The backend is a C++20 HTTP service, and the frontend is a Vue 3 application that exposes interactive operations and experiments, including initialization, insert/find/erase, online resizing, and statistics such as probe counts, fallback occupancy, and kick-depth histograms. Empirical results and engineering observations show that the implementation maintains stable lookup probes at high load factors and remains usable over long runs via online resizing. This work provides a reproducible reference implementation and experimental basis for deploying the theoretical time/space optimal hash table in database-backed and service-oriented settings.

KEYWORDS: Hash table; Time-space tradeoff; Database-backed storage; Fingerprint;

Dynamic Scaling

目 录

目 录	IV
第一章 引言	1
1.1 研究背景与问题动机	1
1.2 研究目标与本文工作	1
1.3 系统实现概览与贡献	2
1.4 论文结构	2
第二章 相关工作与背景	3
2.1 哈希冲突处理与探测代价	3
2.2 时间/空间折中与 OTSH 思想	3
2.3 指纹过滤	4
2.4 数据库化实现的特点	4
第三章 系统总体设计	5
3.1 总体结构	5
3.2 参数与派生规则	5
3.3 映射关系与连续区间	6
3.4 指纹与空槽约定	6
3.5 数据库映射与元数据	6
第四章 核心算法与实现	7
4.1 系统与模块划分	7
4.2 查询算法	7
4.3 插入算法：有界深度 k-kick 与 fallback 兜底	7
4.4 删除算法	8

4.5	在线扩容	8
4.6	HTTP 接口与前端交互	9
第五章 实验设计与结果分析		10
5.1	实验环境与指标	10
5.2	实验一：查询探针与规模关系（probe vs n）	10
5.3	实验二：fallback 使用与负载关系（fallback vs load）	10
5.4	实验三：踢出深度直方图（kick depth histogram）	11
5.5	查询测试：分位探针统计	11
5.6	结果讨论与局限	11
第六章 总结与展望		12
6.1	工作总结	12
6.2	不足与改进方向	12
参考文献		13
致 谢		14
附录 A 接口与实现补充材料		15
A.1	主要 HTTP API	15
A.2	数据库表结构	15
A.3	运行配置	16

第一章 引言

1.1 研究背景与问题动机

哈希表因其高效的键到槽位映射能力，被广泛用于数据库索引、缓存系统、键值存储与网络服务等场景。工程实践中，哈希表通常需要在高负载因子下运行以提升空间利用率，但这会导致冲突概率上升，从而增加查询探测次数、带来不可预测的尾延迟，并放大底层存储或内存层面的访问开销。

在“以探测次数衡量查询代价”的模型下，经典开放定址哈希表在高负载时往往需要多次探测才能完成一次查询；而一些更强的结构（例如通过置换或多选择策略）可以在一定空间开销下显著降低查询探测次数。如何在可控空间代价下，保证查询只需常数次探测，是哈希表理论与工程实现共同关注的问题。

1.2 研究目标与本文工作

本文面向论文《On the Optimal Time/Space Tradeoff for Hash Tables》提出的结构思想^[1]，复现其关键性质并完成工程化实现。该结构的核心在于：将槽位按全局桶（bin）划分，并在桶内进一步划分为若干 mini-bin 与常数大小的回退区（fallback）。对任意键的查询只需访问两个连续下标区间：其偏好 mini-bin 对应的区间，以及同一 bin 的 fallback 区间，从而将一次查询在工程意义上约束为 1 ~ 2 次区间级探测。

为处理插入冲突，本文实现了有界深度的置换插入（k-kick）：当偏好 mini-bin 满时，按确定性规则选择踢出槽位并递归重排，递归深度超过阈值 k 时将元素保留在 fallback 兜底路径，保证插入过程可终止。

1.3 系统实现概览与贡献

本文实现的系统以 MySQL 为持久化存储与并发控制载体^[2]，将逻辑上的一维槽位数组映射为以全局下标为主键的关系表行；后端采用 C++20 构建 HTTP 服务并提供实验接口（基于 `cpp-httpplib`^[3] 与 `nlohmann/json`^[4]），前端采用 Vue 3^[5] 提供交互式操作与可视化展示。本文的主要贡献包括：

- 给出 OTSH 结构在数据库场景下的槽位表示、参数派生与查询/插入/删除流程；
- 实现以“区间扫描次数”为度量的探针统计，支持查询与插入路径的可观测性（`trace` 与统计接口）；
- 设计并实现在线扩容流程，通过双表迁移与原子换表维持系统长期可用性；
- 提供可复现实验接口与前端实验面板，支持探针次数、`fallback` 占比与踢出深度分布等分析。

1.4 论文结构

本文后续章节安排如下：第 2 章介绍相关背景与对比结构；第 3 章给出系统总体设计与参数映射；第 4 章描述核心算法与关键实现细节；第 5 章给出实验设计与结果分析；第 6 章总结全文并展望后续工作。

第二章 相关工作与背景

2.1 哈希冲突处理与探测代价

哈希表的查询与插入性能与冲突处理策略紧密相关。链式哈希通过链表或其他容器存储同一桶内元素，插入较为直接，但在高负载或热点分布下可能产生较长链并引入额外的指针追踪开销。开放定址哈希将元素存放在表内槽位，通过线性探测、二次探测或双重哈希等策略寻找空槽；其优点是空间局部性较好，但在高负载时探测次数会显著上升，并出现聚集效应。

一些更强的结构通过置换或多选择策略控制查询代价。例如 `cuckoo hashing` 通过为每个键提供多个候选位置，并在插入时通过“踢出链”置换已有元素，通常能提供较好的查询上界；`Robin Hood hashing` 则通过均衡探测距离改善尾延迟表现。上述方法在工程中广泛应用，但其查询代价上界、空间开销和实现复杂度之间存在取舍。

2.2 时间/空间折中与 OTSH 思想

理论工作《On the Optimal Time/Space Tradeoff for Hash Tables》^[1]讨论了在空间开销接近线性、负载较高的条件下，如何构造支持常数次探测查询的数据结构。其核心思路之一是改变“探测”的粒度：不再将一次探测限定为单个槽位访问，而是允许一次探测对应一个连续下标区间的扫描（在工程上可对应一次范围查询或一次 `cache-friendly` 扫描）。在这种意义下，如果每次查询只需访问常数个连续区间，即可获得接近常数的查询代价。

本文实现采用的具体结构将槽位划分为 `bin`、`mini-bin` 与 `fallback` 区域，使得每个键的候选位置集中在两个连续区间内，并通过有界深度的置换策略处理冲突。

2.3 指纹过滤

为降低验证成本，工程实现中常采用指纹（**fingerprint**）作为轻量过滤：为每个键计算较短的哈希摘要，查询时先比较指纹，再在必要时比较完整键。本文实现为每个槽位存储 16 位指纹，且约定指纹为 0 表示空槽（当计算结果为 0 时将其映射为 1），以便快速判断占用情况并减少无效比较。

2.4 数据库化实现的特点

与纯内存实现不同，本文将槽位落在 MySQL 表中，利用事务与行锁实现并发安全，并将“区间探测”对应为 SQL 的范围查询。数据库化实现的优点是便于持久化、便于在受限内存环境中进行大规模实验与统计；同时也带来额外的往返开销与事务开销，需要在实现中尽量减少 SQL round-trip（例如在插入中对单个 bin 进行一次锁定并在内存中模拟踢出链，再批量写回）。

第三章 系统总体设计

3.1 总体结构

哈希表在逻辑上视为一维槽位数组 $\text{SLOT}[0 \cdots \text{capacity} - 1]$ 。为控制查询代价并保持空间利用率，本文采用分层组织结构：全局按 bin 划分，再在每个 bin 内划分为若干 mini-bin，并在 bin 尾部设置常数大小的 fallback 区域。整体结构如图 3-1 所示（本文以逻辑示意为主，具体槽位落库见第 3.5 节）。

Bin = MiniBin0 MiniBin1 ... MiniBin(M-1) Fallback

图 3-1 bin / mini-bin / fallback 的逻辑布局示意

3.2 参数与派生规则

系统对外暴露的核心参数包括：目标规模 n 、置换深度上界 k 、负载阈值 load_factor 以及三枚哈希种子。其余尺寸由参数派生得到：

- **mini-bin 尺寸与数量**：根据设计目标， mini_bin_size 与 num_mini_bins 与 $\log^{(k)} n$ 同阶。工程实现中通过“迭代对数”函数计算基值，并对结果进行 clamp（例如限制在 4 到 64）以避免过小导致结构退化或过大导致区间扫描代价上升。
- **fallback 大小**：为常数，本文实现固定为 8。
- **bin 大小**： $\text{bin_size} = \text{mini_bin_size} \times \text{num_mini_bins} + \text{fallback_size}$ 。
- **总 bin 数与容量**： $\text{total_bins} = \left\lceil \frac{n}{\text{bin_size}} \right\rceil$, $\text{capacity_slots} = \text{total_bins} \times \text{bin_size}$ 。
在线扩容将 total_bins 翻倍并更新元数据。

3.3 映射关系与连续区间

对任意键 x ，使用两路哈希分别确定其所属 bin 与偏好 mini-bin:

$$b = h_{\text{bin}}(x) \bmod \text{total_bins}, \quad m = h_{\text{pref}}(x) \bmod \text{num_mini_bins}.$$

在单个 bin 内，以全局连续下标 idx 表示槽位位置。记该 bin 的起始偏移 $\text{offset} = b \times \text{bin_size}$ ，则：

- 偏好 mini-bin 的区间为 $[\text{offset} + m \cdot \text{mini_bin_size}, \text{offset} + (m+1) \cdot \text{mini_bin_size} - 1]$;
- fallback 区间为 $[\text{offset} + \text{mini_bin_size} \cdot \text{num_mini_bins}, \text{offset} + \text{bin_size} - 1]$ 。

因此一次查询只需访问上述两个连续区间即可完成验证，探测次数可按“区间扫描次数”统计（详见第 4.2 节）。

3.4 指纹与空槽约定

每个键 x 计算 16 位指纹 $\text{fp}(x)$ ，用于快速过滤；系统约定 $\text{fp} = 0$ 表示空槽，因此当计算得到 0 时将其映射为 1。槽位是否为空可由 $\text{fp} = 0$ （且 key 为 NULL）判断。

3.5 数据库映射与元数据

本文将逻辑槽位数组映射为数据库表 `hash_table` 的行，使用 `idx` 作为主键，存储 `key` 与 `fp`。为保证键唯一性，在 `key` 上建立唯一约束。元数据表 `ht_meta` 存储 $n, k, \text{load_factor}$ 与种子，并持久化 `total_bins` 以支持在线扩容后的规模维持。

在线扩容采用双表迁移：创建 `hash_table_next` 并按 bin 分步迁移，迁移完成后用 `RENAME TABLE` 原子换表，最后将新的规模写回元数据（详见第 4.5 节）。

第四章 核心算法与实现

4.1 系统与模块划分

后端使用 C++20 实现，HTTP 框架采用 `cpp-httpplib`^[3]，JSON 解析采用 `nlohmann/json`^[4]，并通过 MySQL C API 访问数据库^[2]。实现主要可分为如下模块：哈希函数与指纹（hash）、哈希表操作与参数派生（ht）、数据库封装（db）、在线扩容（resize）以及 HTTP 路由（http_server）。前端使用 Vue 3^[5] + TypeScript 通过 /api 接口调用后端，并提供结构快照与实验结果的可视化展示。

4.2 查询算法

查询目标是在不扫描整表的情况下验证键是否存在。对任意键 x ，首先计算其指纹 $fp(x)$ 以及对应的 b, m ，随后按两段连续区间进行验证：

1. 探测 1：扫描偏好 mini-bin 对应的 idx 区间，若存在 $key=x$ 且 $fp=fp(x)$ 的记录则返回命中；
2. 探测 2：若未命中，则扫描同一 bin 的 fallback 区间，若命中则返回；否则返回未找到。

在实现中，probes 定义为“区间扫描次数”，每执行一次范围查询（idx BETWEEN start AND end）则 probes 加 1。因此一次查询的 probes 在结构意义上被约束为 2（无并发扩容与异常情况时）。

4.3 插入算法：有界深度 k-kick 与 fallback 兜底

插入包含两层目标：尽可能把键放入偏好 mini-bin（便于查询只扫描两段区间），并在冲突时通过有限步置换腾挪空间。本文实现的插入策略如下：

- **优先填充**：若偏好 mini-bin 存在空槽，直接写入该槽；
- **置换链**：若偏好 mini-bin 满，执行深度不超过 k 的踢出过程。与在数据库中随机选择踢出槽不同，本文使用与 fp、深度与种子相关的确定性函数选择踢出位置，以避免 ORDER BY RAND() 等高代价操作；
- **fallback 兜底**：当置换深度超过 k 或置换无法继续时，将当前被踢出的元素（或最终元素）放入该 bin 的 fallback 区域；若 fallback 已满，则插入失败。

为减少数据库往返，插入实现采用“bin 级锁定 + 内存模拟 + 批量写回”的方式：在事务中一次性锁定目标 bin 的所有槽位（一次范围查询），在内存中模拟踢出链并记录变更槽集合，最后用 UPDATE ... CASE 将变更批量写回。该策略在保证正确性的同时，显著减少了踢出链中逐槽 SELECT FOR UPDATE 的开销。

4.4 删除算法

删除操作首先通过唯一约束在 key 上定位 idx，若存在则把该槽位更新为空（key=NULL，fp=0）。删除本身不触发重排；在高负载环境下，可通过再插入或扩容恢复局部空槽分布。

4.5 在线扩容

在线扩容旨在当负载超过阈值时扩大表容量并保持接口可用。本文实现采用双表迁移策略：

1. 触发：批量插入前估算插入结束时负载，若预计超过阈值，则启动扩容；
2. 创建 next 表：构建 hash_table_next，行数为 new_total_bins × bin_size，并批量写入空槽；
3. 分步迁移：以 bin 为单位迁移每个旧 bin 中的非空键到 next 表（每步一个短事务），更新进度指标供前端展示；
4. 原子换表：迁移完成后使用 RENAME TABLE 将 next 表交换为主表，并删除旧表；

5. 持久化规模：将 `total_bins` 与名义容量 (`ht_meta.n`) 写回元数据。

该实现将扩容进度与耗时统计保存在进程内状态中，并通过统计接口返回给前端，用于显示扩容进度与耗时。

4.6 HTTP 接口与前端交互

后端对外提供如下典型接口：初始化 (`POST /api/init`)、单键操作 (`/api/insert`, `/api/find`, `/api/erase`)、统计与可视化 (`GET /api/stats`, `/api/snapshot`, `/api/bins`) 以及实验接口 (`/api/query_test` 与 `/api/experiment/*`)。前端提供初始化表、批量插入、查询测试与实验运行等操作入口，并将结构快照与实验曲线以图表方式展示。

接口的具体字段与附录中的 JSON 示例保持一致（见附录）。

第五章 实验设计与结果分析

5.1 实验环境与指标

本文实现以数据库为存储与计算载体，实验重点关注“区间扫描次数”意义下的探针代价，并统计结构在不同负载与参数设置下的行为。主要指标包括：

- **探针次数**：查询的 probes，定义为对连续 idx 区间进行一次范围扫描计 1 次；
- **分位探针**：在多次查询测试中统计平均探针、P99 探针与最大探针；
- **fallback 占比**：fallback 区域已用槽数及其占已用槽的比例；
- **踢出深度分布**：插入过程中 place/kick 的深度直方图，用于观察 k 对置换链的影响。

5.2 实验一：查询探针与规模关系（probe vs n ）

该实验通过 `POST /api/experiment/probe_vs_n` 对多组 n 与 k 重复执行“init + 插入 + 查询测试”，输出每个点的平均探针、P99 探针与最大探针。该实验是破坏性的，会反复重建表。

预期现象：在固定 load factor 与结构规则下，查询探针受两段区间扫描上界约束，平均探针应接近常数；不同 k 影响插入时落入 fallback 的比例，从而可能间接影响查询命中分布与探针统计。

5.3 实验二：fallback 使用与负载关系（fallback vs load）

该实验通过 `POST /api/experiment/fallback_vs_load` 将表逐步填充至多个目标负载（如 0.7、0.8、0.9、0.95、0.98），统计每个阶段的实际 load、fallback 已用槽与 fallback 占比。

预期现象：随着负载上升，偏好 mini-bin 更容易被填满，插入置换链更频繁，fallback 作为兜底路径的使用比例上升；在较小 k 或偏斜分布下，fallback 占比上升更明显。

5.4 实验三：踢出深度直方图 (kick depth histogram)

该实验通过 GET /api/experiment/kick_depth_hist 返回插入过程中记录的深度直方图（在 /api/init 时清空）。其中 depth=0 表示直接放入空槽，depth>0 表示发生踢出链；额外桶用于统计“仅落入 fallback”的事件。

预期现象：在合理参数下，踢出深度主要集中在较小深度；增大 k 会延长允许的置换链，减少落入 fallback 的事件，但可能增加插入代价。

5.5 查询测试：分位探针统计

通过 POST /api/query_test 可在给定查询次数与命中率配置下执行大量查询，并返回平均探针、P99 与最大探针。该接口可以用于在固定表状态下观察命中/未命中混合查询的尾部行为。

5.6 结果讨论与局限

由于本章目标是形成可编译的论文初稿，本文在初稿阶段先给出实验设计、指标定义与预期现象，并与系统已实现的实验接口一一对齐。后续可直接运行前端实验面板或调用相应 API，导出真实曲线与数据表格，并将本章的“预期现象”替换为“实测结果与分析”。

第六章 总结与展望

6.1 工作总结

本文围绕《On the Optimal Time/Space Tradeoff for Hash Tables》提出的结构思想，完成了其在工程环境中的复现与系统化实现。本文实现通过 bin/mini-bin/fallback 的分层布局，使得查询仅需访问偏好 mini-bin 与 fallback 两段连续下标区间，从而在“区间扫描次数”意义下将查询探针限制为常数。插入方面，实现了有界深度的 k-kick 置换策略与 fallback 兜底路径，并在数据库化场景下通过“锁定单个 bin + 内存模拟踢出链 + 批量写回”的方式减少 SQL 往返。系统层面，本文提供了完整的 HTTP 接口与前端实验面板，并实现在线扩容流程以支撑高负载下的长期运行与可视化分析。

6.2 不足与改进方向

数据库化实现在可复现与可观测方面具有优势，但也带来额外开销与工程约束。后续可从以下方向改进：

- **性能优化**: 进一步减少范围查询与事务开销，例如更细粒度的批量读写、更激进的缓存策略或预取；
- **并发与隔离**: 在扩容期间的读写隔离、跨线程与跨进程的扩容状态持久化，以及更稳健的失败恢复；
- **实验完善**: 补齐不同分布 (uniform/偏斜) 与不同参数 (k 、load factor、mini-bin 派生规则) 下的全面对比，并与经典结构进行对照实验；
- **功能扩展**: 支持更多键值类型、批量删除与重建策略，以及更丰富的可视化与数据导出能力。

参考文献

- [1] PAGH R, SEIDEL T, WIEDER U. On the Optimal Time/Space Tradeoff for Hash Tables[J]. Proceedings of the ACM on Management of Data, 2021.
- [2] Oracle. MySQL Reference Manual[A/OL]. 2026 [2026-04-27]. <https://dev.mysql.com/doc/>.
- [3] HIRAOKA Y. cpp-httpplib[EB/OL]. 2026 [2026-04-27]. <https://github.com/yhirose/cpp-httpplib>.
- [4] LOHMANN N. JSON for Modern C++[EB/OL]. 2026 [2026-04-27]. <https://github.com/nlohmann/json>.
- [5] Vue.js Contributors. Vue.js Documentation[EB/OL]. 2026 [2026-04-27]. <https://vuejs.org/>.

致 谢

感谢 LUG@NJU。

附录 A 接口与实现补充材料

A.1 主要 HTTP API

本文系统通过 HTTP 提供操作与实验接口，典型接口如下（具体字段以实现为准）：

- **健康检查**: GET /health, 返回 ok;
- **初始化**: POST /api/init, 请求体包含 `n`, `k`, `load_factor` (可选 `seed1/2/3`), 返回派生参数 (`mini_bin_size`, `num_mini_bins`, `bin_size`, `total_bins`, `capacity_slots` 等);
- **单键插入**: POST /api/insert, 请求体 {`key`, `trace?`}, 返回 ok, `probes`, `fp`, `bin`, `mini`, 当 `trace=true` 时返回踢出链步骤;
- **单键查询**: POST /api/find, 请求体 {`key`}, 返回 found, `probes`;
- **单键删除**: POST /api/erase, 请求体 {`key`}, 返回 ok, `probes`;
- **统计**: GET /api/stats, 返回 `used_slots`, `fallback_used`, `capacity_slots`, `load_factor`, 以及扩容状态摘要;
- **bin 统计**: GET /api/bins?start=...&count=...;
- **结构快照**: GET /api/snapshot?bin_start=...&bin_count=...;
- **查询测试**: POST /api/query_test, 返回 `avg_probes`, `p99_probes`, `max_probes`;
- **实验接口**: /api/experiment/kick_depth_hist、/api/experiment/probe_vs /api/experiment/fallback_vs_load。

A.2 数据库表结构

主槽位表使用全局下标作为主键，空槽用 `fp=0` 与 `key=NULL` 表示；`key` 列具有唯一约束以避免重复键。

```
CREATE TABLE hash_table (  
    idx BIGINT PRIMARY KEY,  
    key BIGINT UNSIGNED NULL,  
    fp SMALLINT UNSIGNED NOT NULL,  
    UNIQUE KEY uk_key (key)  
);
```

A.3 运行配置

后端通过环境变量读取数据库连接信息与服务端口等配置,例如:DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, 服务端口由 PORT 指定; 可选 LOG_FILE 输出日志文件。